

Chương 2: CÁC THUẬT TOÁN XỬ LÝ ẢNH (PHẦN 02)

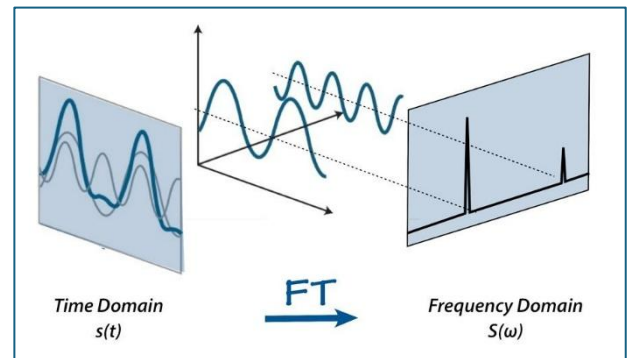
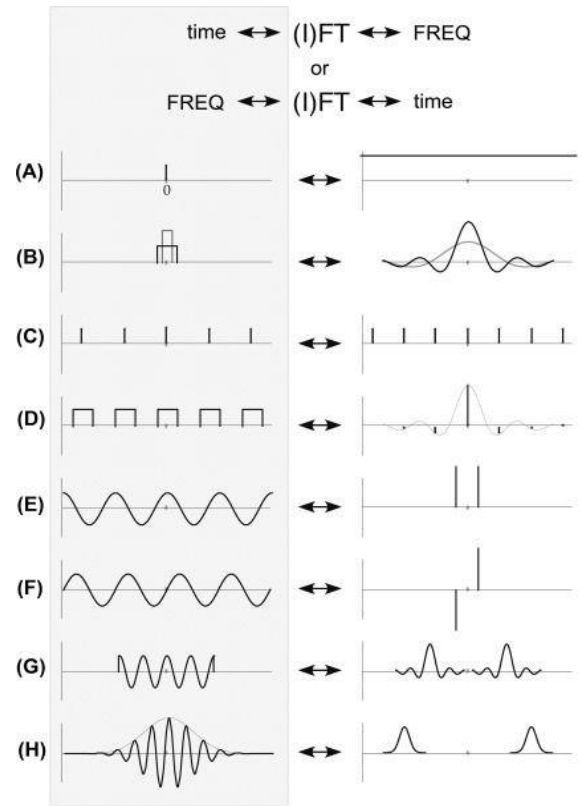
Nội dung

1. Biến đổi Fourier.....	2
1.1. Biến đổi Fourier là gì?	2
1.2. Mục đích sử dụng biến đổi Fourier	3
1.3. Ý nghĩa vật lý của biến đổi Fourier	4
1.4. Các loại biến đổi Fourier	5
1.5. Ứng dụng của biến đổi Fourier	6
1.6. Biến đổi Fourier trong xử lý ảnh	7
2. Biến đổi Wavelet	9
2.1. Biến đổi Wavelet trong xử lý ảnh.....	10
2.2. Ứng dụng của Biến đổi Wavelet.....	11
2.3. Một số loại wavelet phổ biến.....	11
2.4. Quy trình thực hiện biến đổi Wavelet	12
2.5. Biến đổi Wavelet rời rạc 2D (2D-DWT)	12
3. Biến đổi hình học trong xử lý ảnh (Geometric Transformations)	18
3.1. Các loại biến đổi hình học phổ biến.....	18
3.2. Ma trận biến đổi	22
3.3. Phương pháp Nội suy.....	22
3.4. Biến đổi hình học trong xử lý ảnh với thư viện OpenCV và Pillow	23
4. Xử lý hình thái học (Morphology)	24

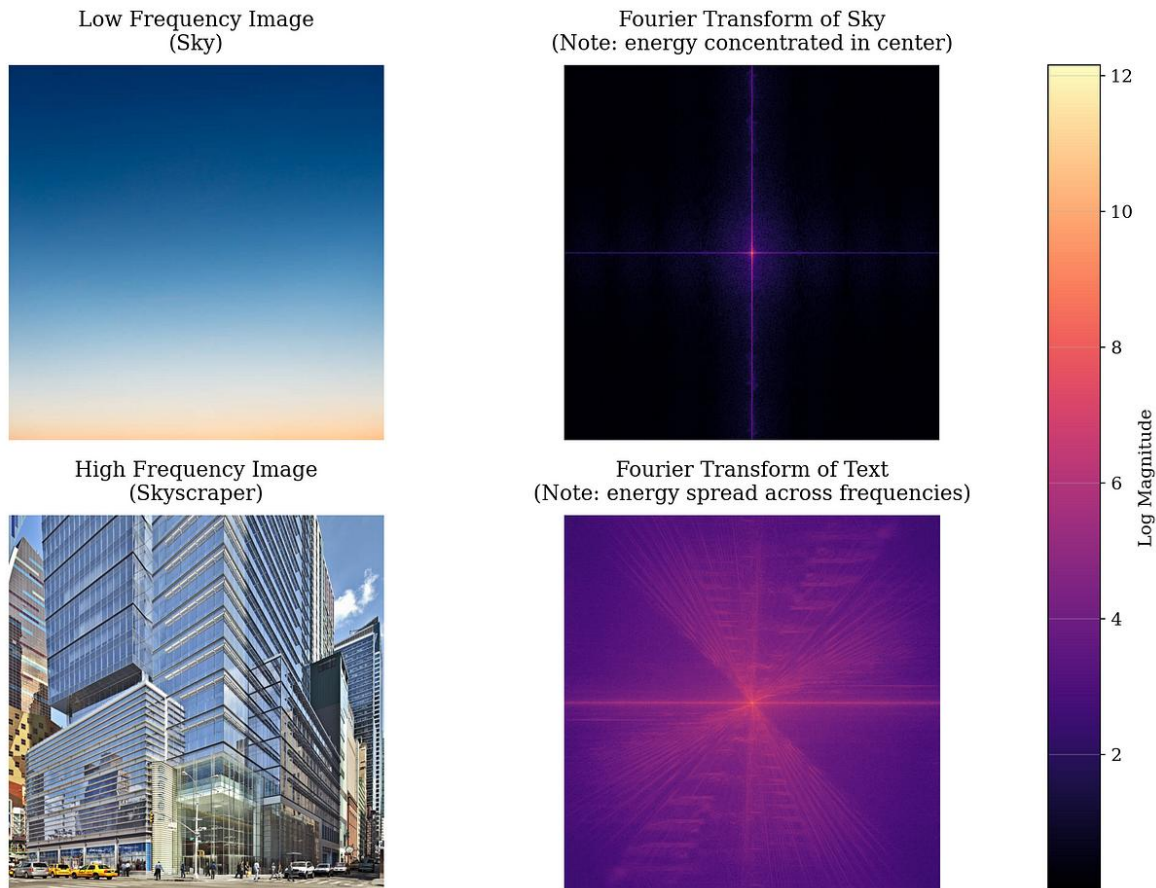
1. Biến đổi Fourier

1.1. Biến đổi Fourier là gì?

- Sử dụng rộng rãi trong nhiều lĩnh vực như xử lý tín hiệu, vật lý, kỹ thuật điện, và thậm chí cả khoa học máy tính.
- Giúp chuyển đổi một tín hiệu từ **miền thời gian** (miêu tả sự thay đổi của tín hiệu theo thời gian) sang **miền tần số** (miêu tả các thành phần tần số cấu tạo nên tín hiệu đó)
 - Mọi tín hiệu (âm thanh, hình ảnh) dù phức tạp đến đâu cũng có thể được tạo thành bằng cách **cộng gộp các sóng sin và cos đơn giản có tần số khác nhau**.
- Miền thời gian (Time Domain):** Biểu diễn *tín hiệu thay đổi thế nào theo thời gian* (ví dụ: biên độ âm thanh).
 - Trong ảnh, đây là miền không gian (Spatial Domain), biểu diễn **giá trị pixel tại tọa độ (x, y)**.
- Miền tần số (Frequency Domain):** Biểu diễn tín hiệu đó được **cấu tạo từ bao nhiêu sóng sin/cos**, với tần số và biên độ bao nhiêu.
 - 1D:** Tần số là tốc độ thay đổi của tín hiệu theo *thời gian* (Hz).
 - Trong xử lý ảnh, khái niệm "thời gian" biến mất và được thay thế bằng "**không gian**" (Spatial). Lúc này, "tần số" không phải là độ rung nhanh hay chậm nữa, mà là **độ thay đổi màu sắc/độ sáng** giữa các điểm ảnh nằm cạnh nhau.
 - Tần số là tốc độ thay đổi độ sáng** (mức xám) **theo khoảng cách** (không gian). Chúng ta gọi là **Tần số không gian (Spatial Frequency)**.
 - Tần số thấp:** Là những vùng mà màu sắc hoặc độ sáng thay đổi **từ từ, êm dịu**. Ở đó, *điểm ảnh này và điểm ảnh bên cạnh nó có màu gần như y hệt nhau*.
 - Tần số cao:** Là những vùng mà màu sắc thay đổi **đột ngột, gay gắt**. Ở đây, *điểm ảnh đang sáng bỗng nhiên chuyển sang tối ngay lập tức*.
- Biến đổi Fourier giống như việc bạn có một bộ lọc đặc biệt. Nó giúp bạn tách riêng "phần cốt lõi, mượt mà" của bức ảnh ra một bên và "phần chi tiết, gai góc" ra một bên.**
 - Khi áp dụng biến đổi Fourier lên một bức ảnh, kết quả nhận được không phải là một bức ảnh chân dung hay phong cảnh nữa, mà là một **bản đồ tần số** (Spectrum).



- Trên bản đồ này, thay vì thấy "mắt, mũi, miệng", bạn sẽ thấy sự phân bố của các tần số:
 - **Vùng trung tâm** bản đồ thường đại diện cho **tần số thấp** (những vùng màu mượt mà, ít thay đổi).
 - **Vùng rìa ngoài (peripheral)** đại diện cho **tần số cao** (những chi tiết sắc nét, cạnh viền, nhiễu).
- **Biến đổi Fourier** giống như việc bạn có một bộ lọc đặc biệt. Nó giúp bạn tách riêng "phần cốt lõi, mượt mà" của bức ảnh ra một bên và "phần chi tiết, gai góc" ra một bên. *Ta có quyền giữ lại một trong hai phần này.*



1.2. Mục đích sử dụng biến đổi Fourier

- **Phân tích phổ tần:** Khi ta phân tích một tín hiệu phức tạp thành các thành phần tần số đơn giản, ta có thể **hiểu rõ hơn về bản chất của tín hiệu đó**. Ví dụ, trong âm nhạc, biến đổi Fourier giúp ta phân tích một nốt nhạc thành các sóng hài, từ đó xác định được cao độ và âm sắc của nốt nhạc đó.
- **Lọc tín hiệu:** Bằng cách **loại bỏ các thành phần tần số không mong muốn**, ta có thể lọc sạch một tín hiệu, giảm nhiễu và cải thiện chất lượng tín hiệu.
- **Nén dữ liệu:** Biến đổi Fourier giúp ta nén dữ liệu bằng cách **loại bỏ các thành phần tần số có biên độ nhỏ**, không ảnh hưởng nhiều đến chất lượng tín hiệu.
- **Giải phương trình vi phân:** Biến đổi Fourier giúp chuyển đổi các phương trình vi phân thành các phương trình đại số dễ giải hơn.

1.3. Ý nghĩa vật lý của biến đổi Fourier

- Mỗi tín hiệu đều có thể được biểu diễn dưới dạng tổng của các hàm sin và cos với các tần số và biên độ khác nhau. Đây là một khái niệm cốt lõi trong biến đổi Fourier.
 - **Biên độ** của mỗi thành phần tần số cho biết mức độ đóng góp của tần số đó vào tín hiệu.
 - **Cho biết độ mạnh/yếu của thành phần tần số đó.**
 - Các thành phần tần số có biên độ lớn đóng góp nhiều hơn vào tín hiệu so với các thành phần có biên độ nhỏ.
 - **Trong ảnh:** Nó thể hiện **độ tương phản (contrast)** của các **gợn sóng sáng tối** đó.
 - **Biên độ lớn:** Sự chênh lệch giữa vùng sáng và vùng tối của cái "sọc" đó rất rõ rệt (trắng tinh vs đen tuyền).
 - **Biên độ nhỏ:** Sự chênh lệch mờ nhạt (xám sáng vs xám tối).
 - **Pha** của mỗi thành phần tần số cho biết *sự dịch chuyển pha của thành phần đó so với một hàm sin hoặc cos chuẩn*. Pha cung cấp thông tin về mối quan hệ thời gian giữa các thành phần tần số khác nhau.
 - **Thông tin về sự dịch chuyển hay vị trí.**
 - **Trong ảnh:** Pha chứa đựng thông tin về **vị trí của các đường nét và hình dạng**.
 - **VD:** video chỉ vật đi từ A → B trong vòng 1s, và 30fps
 - Frame ảnh đầu, vật tại vị trí A
 - Frame ảnh cuối, vật tại vị trí B
- ⇒ Có thể hiểu "**pha ảnh**" là "**những frame ảnh còn lại = 28 frames**". Để mô tả việc các frame này cho thấy sự thay đổi vị trí của vật theo thời gian, có thể dùng "**quá trình chuyển động được thể hiện qua các frames**" thay cho "**pha ảnh**".

1.4. Các loại biến đổi Fourier

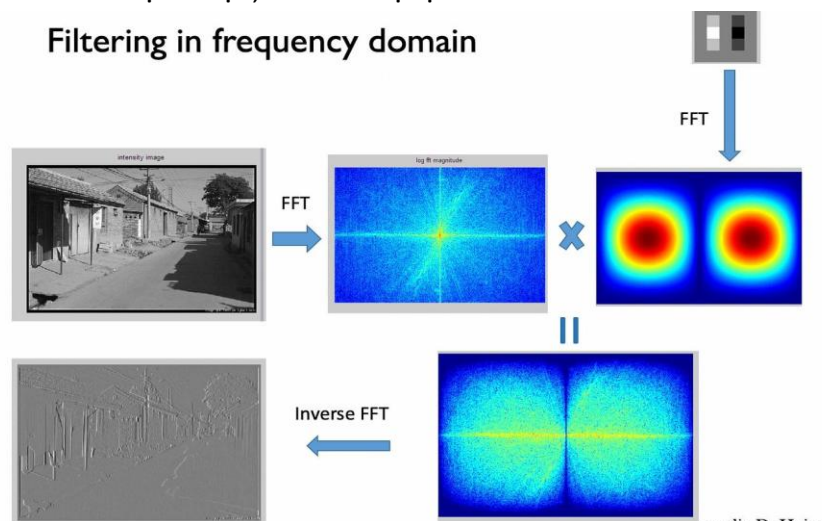
Đặc điểm	Biến đổi Fourier Liên tục (Continuous Fourier Transform - CFT)	Biến đổi Fourier Rời rạc (Discrete Fourier Transform - DFT)	Biến đổi Fourier Nhanh (Fast Fourier Transform - FFT)
Áp dụng	Áp dụng cho các tín hiệu liên tục theo thời gian. Thường được sử dụng trong lý thuyết và phân tích tín hiệu lý tưởng.	Áp dụng cho các tín hiệu rời rạc, được lấy mẫu ở các thời điểm rời rạc. Máy tính không hiểu liên tục, nó chỉ hiểu các điểm rời rạc (pixel). Phù hợp cho hầu hết các ứng dụng thực tế, đặc biệt khi xử lý tín hiệu số.	Một thuật toán hiệu quả để tính toán biến đổi Fourier rời rạc. Đây là một thuật toán tối ưu để tính DFT cực nhanh ($O(N\log N)$). Đây là thuật toán nền tảng của mọi công nghệ xử lý số hiện đại (MP3, JPEG, WiFi...). Luôn được ưu tiên sử dụng khi cần tính toán DFT một cách nhanh chóng và hiệu quả. Trong lập trình (Python/OpenCV), khi gọi hàm Fourier, thực chất máy tính đang chạy FFT.
Tín hiệu & Phạm vi tần số	Liên tục	Rời rạc	Rời rạc
Tính toán	Tích phân	Tổng	Thuật toán hiệu quả
Ưu điểm	Toàn diện, lý thuyết	Thực tế, tính toán nhanh	Rất nhanh
Hạn chế	Ít thực tế, tính toán phức tạp	Aliasing, giới hạn độ phân giải	-

Type of Transform	Example Signal
Fourier Transform <i>signals that are continuous and aperiodic</i>	
Fourier Series <i>signals that are continuous and periodic</i>	
Discrete Time Fourier Transform <i>signals that are discrete and aperiodic</i>	
Discrete Fourier Transform <i>signals that are discrete and periodic</i>	

1.5. Ứng dụng của biến đổi Fourier

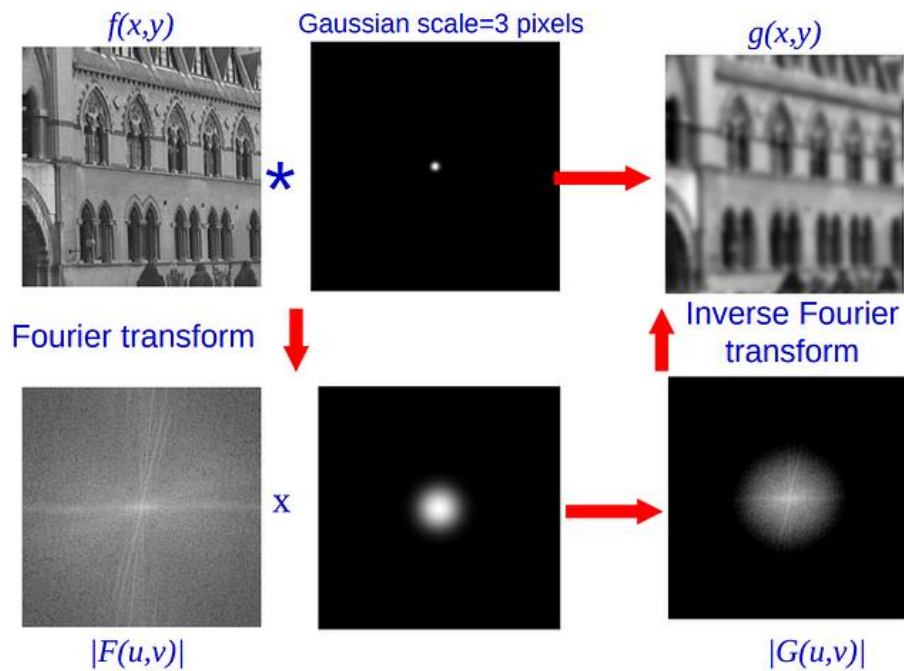
- Xử lý âm thanh: Lọc nhiễu, nén âm thanh, nhận dạng giọng nói.
- Xử lý ảnh: Lọc ảnh, nén ảnh, tăng cường cạnh.
- Xử lý tín hiệu sinh học: Phân tích tín hiệu điện não đồ (EEG), điện tâm đồ (ECG).
- Truyền thông: Điều chế tín hiệu, thiết kế bộ lọc.
- Vật lý: Phân tích phổ, giải phương trình sóng.
- Kỹ thuật điện: Phân tích mạch điện, thiết kế bộ lọc.

Filtering in frequency domain

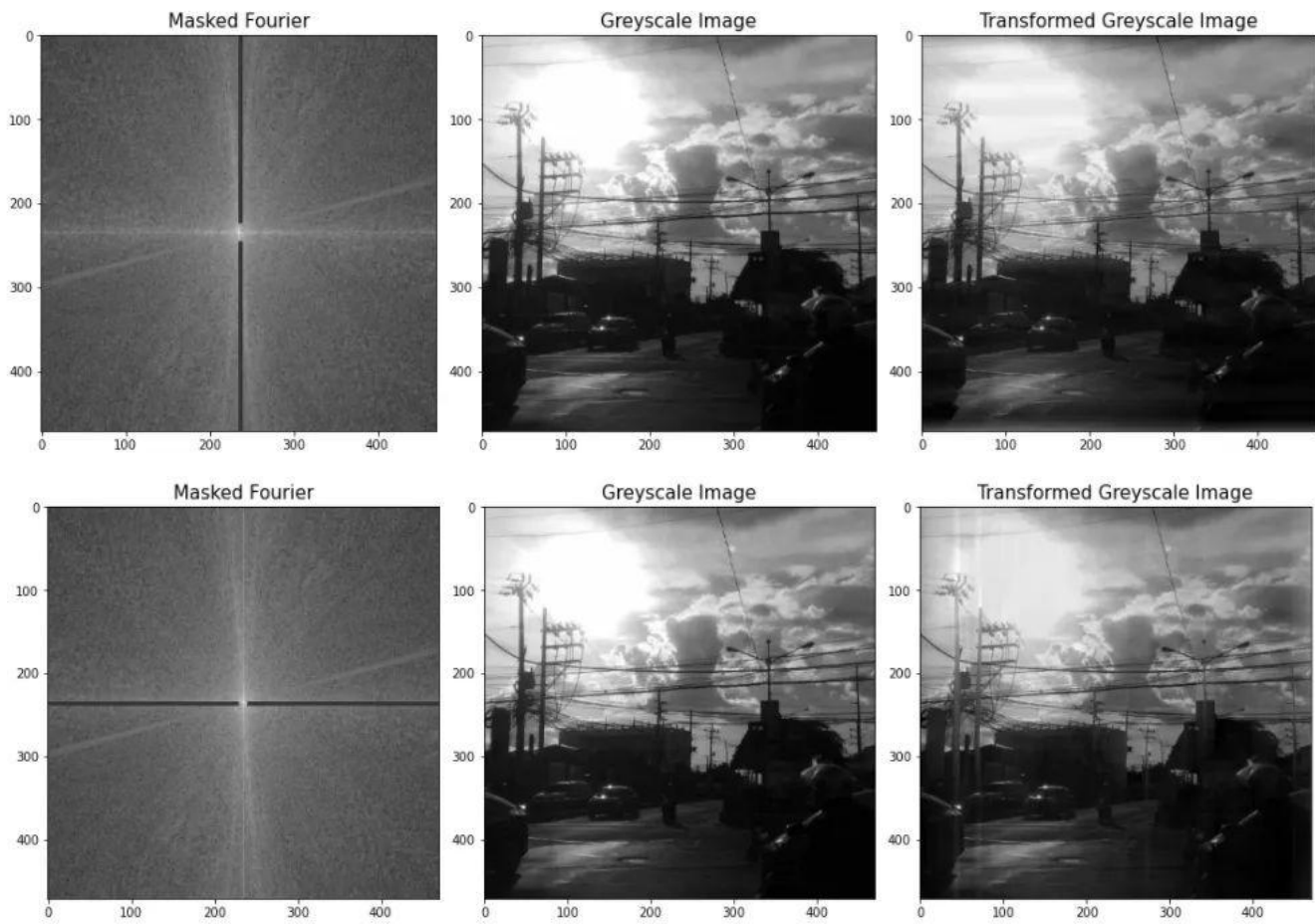


1.6. Biến đổi Fourier trong xử lý ảnh

- Khi áp dụng biến đổi Fourier vào một bức ảnh, chúng ta sẽ **chuyển đổi ảnh từ miền không gian (pixel) sang miền tần số**. Điều này giúp chúng ta phân tích cấu trúc tần số của ảnh, từ đó thực hiện các phép biến đổi và xử lý ảnh hiệu quả hơn.
- **Mục đích sử dụng biến đổi Fourier trong xử lý ảnh**
 - Phân tích cấu trúc tần số: Các thành phần **tần số thấp** tương ứng với các chi tiết lớn, vùng sáng tối trong ảnh. Các thành phần **tần số cao** tương ứng với các chi tiết nhỏ, cạnh, nhiễu.
 - **Lọc ảnh: Loại bỏ nhiễu** bằng cách loại bỏ các thành phần tần số cao. Tăng cường cạnh bằng cách **tăng cường các thành phần tần số cao**.
 - **Nén ảnh: Bỏ qua các thành phần tần số có biên độ nhỏ**, không ảnh hưởng nhiều đến chất lượng ảnh.
 - Nhận dạng mẫu: Phân tích phổ tần của các mẫu đặc trưng để tìm kiếm sự tương đồng.
- **Phân tích phổ**: Giúp ta biết trong bức ảnh, chỗ nào thay đổi nhanh (tần số cao - thường là cạnh, nhiễu), chỗ nào thay đổi chậm (tần số thấp - vùng màu mượt).
- **Lọc tín hiệu**: Trong miền tần số, việc lọc rất dễ. Muốn xóa nhiễu (thường là tần số cao)? Chỉ cần cắt bỏ phần tần số cao đi.
- **Các ứng dụng cụ thể**
 - **Lọc ảnh**:
 - Lọc thông thấp: Chỉ cho tần số thấp đi qua → Ảnh bị mờ đi (Blur), mất chi tiết cạnh, nhưng khử được nhiễu hạt → Làm mờ ảnh, loại bỏ nhiễu.
 - Lọc thông cao: Chỉ cho tần số cao đi qua → Giữ lại cạnh, làm nổi bật chi tiết, nhưng vùng mượt sẽ bị đen → Tăng cường cạnh, chi tiết.
 - Lọc băng thông: **Kết hợp cả hai loại trên** để loại bỏ một số tần số nhất định.
 - **Nén ảnh** với ảnh JPEG, sử dụng biến đổi cosin rời rạc (DCT), một biến thể của DFT.
 - **Nhận dạng mẫu**: Phân tích phổ tần của các mẫu vân tay, khuôn mặt để xác thực.
 - **Xử lý ảnh y tế**: Phân tích hình ảnh X-quang, MRI để phát hiện các bất thường.
- **Quy trình thực hiện biến đổi Fourier**
 - **Biến đổi Fourier 2D**: Áp dụng biến đổi Fourier cho từng hàng và từng cột của ma trận ảnh.
 - **Phân tích phổ tần**: Quan sát phổ tần để hiểu cấu trúc tần số của ảnh.
 - **Thực hiện các phép biến đổi**: Lọc, tăng cường, nén,...
 - **Biến đổi Fourier ngược**: Chuyển đổi ảnh trở lại miền không gian.



- Lọc trong miền tần số (Frequency Domain Filtering).**



- Hai bức ảnh này minh họa cách chúng ta có thể "xóa" các chi tiết cụ thể trong ảnh gốc bằng cách **can thiệp vào bản đồ Fourier của nó**.

- **Greyscale Image (Ảnh xám gốc):** Đây là ảnh đầu vào. **Bạn hãy chú ý đến hai chi tiết chính: các cột điện (thẳng đứng) và các dây điện (nằm ngang).**
- **Masked Fourier (Phổ Fourier đã bị che):** Đây là bản đồ tần số của bức ảnh, nhưng đã bị một vạch đen che mất một phần. **Vạch đen này đóng vai trò là "cục tẩy", xóa bỏ toàn bộ năng lượng tần số tại vùng đó (cho về 0).**
- **Transformed Greyscale Image (Ảnh kết quả):** Đây là bức ảnh được tái tạo lại sau khi đã bị "tẩy" mất một số tần số.
- **Phân tích chi tiết "Masked Fourier":** Quy tắc vàng để nhớ trong biến đổi Fourier 2D là: **"Trục này quản lý chiều kia".**
 - **Trục Ngang (trong miền tần số):** Chứa thông tin **về các mẫu thay đổi theo chiều ngang** → Tương ứng với các chi tiết **dạng sọc đứng** (như cột điện).
 - **Trục Dọc (trong miền tần số):** Chứa thông tin **về các mẫu thay đổi theo chiều dọc** → Tương ứng với các chi tiết **dạng sọc ngang** (như dây điện).
- **Ảnh 1 (Vạch đen thẳng đứng)**
 - **Hành động:** Chúng ta kẻ một vạch đen đè lên trục dọc của phổ Fourier.
 - **Ý nghĩa:** Chúng ta đang xóa bỏ các tần số quy định các chi tiết nằm ngang.
 - **Kết quả:** Hãy nhìn vào ảnh bên phải cùng. Lần này, các **dây điện** (nằm ngang) đã biến mất! Chỉ còn lại các cột điện trơ trọi.
- **Ảnh 2 (Vạch đen nằm ngang)**
 - **Hành động:** Chúng ta kẻ một vạch đen đè lên trục ngang của phổ Fourier.
 - **Ý nghĩa:** Chúng ta đang xóa bỏ các tần số quy định các chi tiết thẳng đứng.
 - **Kết quả:** Hãy nhìn vào ảnh bên phải cùng. Bạn có thấy các **cột điện** (thẳng đứng) đã bị mờ đi rất nhiều hoặc biến mất không? Trong khi đó, các dây điện (nằm ngang) vẫn còn khá rõ.

2. Biến đổi Wavelet

- Phép biến đổi wavelet phân tích tín hiệu thành wavelet.
- Wavelet là **các sóng nhỏ với tần số và thời lượng khác nhau**, khiến chúng trở nên lý tưởng để chụp cả các đặc điểm toàn cục và cục bộ trong một hình ảnh.
- **Tại sao cần biến đổi Wavelet?**
 - **Phân tích đa tỷ lệ:** Wavelet cho phép ta phân tích tín hiệu ở nhiều mức độ chi tiết khác nhau, từ các đặc trưng tổng quát đến các chi tiết cục bộ. Điều này rất hữu ích khi phân tích các tín hiệu có cả thành phần tần số thấp và cao.
 - **Local hóa trong cả thời gian và tần số:** Khác với Fourier chỉ phân tích tín hiệu trong miền tần số (Toàn cục), wavelet cho phép ta **xác định cả vị trí và tần số của các đặc trưng** trong tín hiệu (Cục bộ).

- Hãy tưởng tượng ví dụ về bản nhạc:
 - **Fourier:** Ông nhạc trưởng nghe hết cả bài nhạc dài 5 phút, sau đó đưa cho bạn một danh sách: "Trong bài này có nốt Đồ, nốt Mi, và nốt Sol".
 - Vấn đề: Bạn hỏi: "Thế nốt Sol xuất hiện ở phút thứ mấy? Lúc bắt đầu hay lúc kết thúc?". Ông ấy trả lời: "Tôi không biết, tôi chỉ biết là có nốt đó trong bài thôi".
 - **Mất thông tin về thời gian (vị trí).**
 - **Wavelet:** Người biên tập viên này thông minh hơn. Anh ta nói: "Nốt Sol xuất hiện ở giây thứ 10, kéo dài 2 giây, rồi biến mất. Sau đó nốt Đồ xuất hiện ở phút thứ 3".
 - **Biết rõ Tần số (nốt nhạc) VÀ Thời gian (khi nào nó xuất hiện).**
- **Phù hợp với tín hiệu không dừng**, tức là các tín hiệu có đặc trưng thay đổi theo thời gian.

Đặc điểm	Biến đổi Fourier	Biến đổi Wavelet
Cửa sổ phân tích	Cửa sổ có độ dài cố định	Cửa sổ có độ dài thay đổi theo tần số
Tính chất	Tính toán nhanh (FFT)	Linh hoạt hơn
Ứng dụng	Phân tích phổ, lọc tín hiệu	Phân tích đa tỷ lệ, nén dữ liệu, xử lý ảnh

2.1. Biến đổi Wavelet trong xử lý ảnh

- Phép biến đổi Fourier chỉ giới hạn ở tần số, trong khi đó Wavelet **xem xét cả thời gian và tần suất**. Biến đổi này phù hợp với các tín hiệu không cố định.
- Cạnh là một trong những phần quan trọng của hình ảnh, tuy nhiên, khi áp dụng các bộ lọc truyền thống, nhiễu được loại bỏ nhưng hình ảnh bị mờ, do Nhiễu (hạt lấm tấm) và Cạnh (đường viền vật thể) đều là **tần số cao** (thay đổi đột ngột). Khi dùng Fourier hay bộ lọc làm mờ (Blur) để xóa nhiễu, máy tính "lỡ tay" xóa luôn cả cạnh, làm ảnh bị nhòe.
- **Biến đổi Wavelet được thiết kế để có được độ phân giải tần số tốt cho các thành phần tần số thấp.**
 - Vì Wavelet biết **vị trí**, nó có thể phân biệt được đâu là "nhiều lung tung" và đâu là "cạnh quan trọng". Nó cho phép làm sạch nhiễu ở những vùng phẳng lặng mà vẫn giữ nguyên độ sắc nét ở các đường viền.
- Wavelet cho phép **phân tích một hình ảnh thành các thành phần ở các độ phân giải khác nhau**, giúp nắm bắt được cả các đặc trưng toàn cục (ví dụ như đường nét lớn) và các chi tiết cục bộ (ví dụ như cạnh, góc) của hình ảnh.
- **Hệ số wavelet.** Khi áp dụng biến đổi wavelet lên một hình ảnh, ta sẽ **thu được một ma trận các hệ số wavelet**, mỗi hệ số đại diện cho một mức độ tương quan giữa hình ảnh và một wavelet con cụ thể.
 - Hãy tưởng tượng bạn có một **khuôn mẫu hình "lượn sóng" nhỏ** (gọi là wavelet mẹ). Bạn cầm khuôn mẫu này rà đi rà lại trên bức ảnh.
 - **Hệ số lớn:** Chỗ nào trên ảnh giống y hệt cái khuôn mẫu đó → ghi lại số to.
 - **Hệ số nhỏ/bằng 0:** Chỗ nào không giống → ghi số nhỏ.

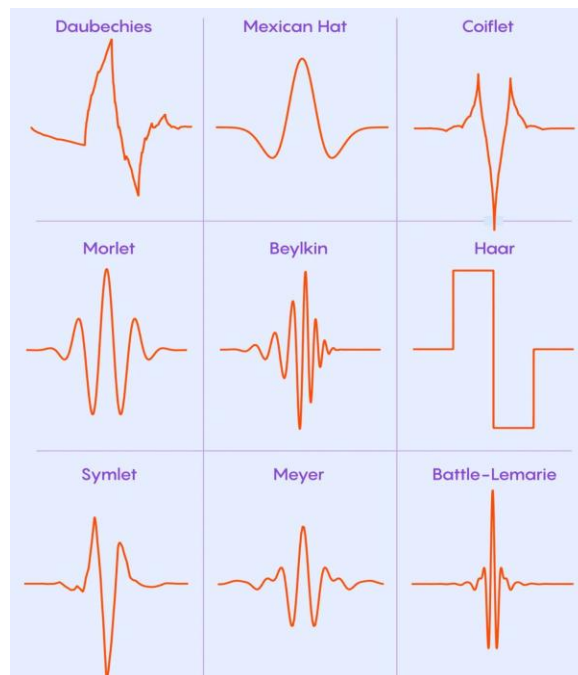
- Kết quả là một bảng số (ma trận) cho biết **mức độ "giống" của từng vùng ảnh với cái khuôn mẫu chuẩn.**
- **Phân giải.** Mỗi cấp độ phân giải trong biến đổi wavelet tương ứng với một độ phân giải khác nhau của hình ảnh.

2.2. Ứng dụng của Biến đổi Wavelet

- **Xử lý ảnh:**
 - Nén ảnh: Wavelet cho phép nén ảnh với chất lượng cao hơn so với các phương pháp khác.
 - Phân tích đặc trưng (Feature Extraction): Tìm kiếm các đặc trưng như cạnh, góc, kết cấu trong ảnh (Wavelet transforms can be used to extract important features to improve neural network's performance).
 - Loại bỏ nhiễu: Loại bỏ các thành phần nhiễu ở các mức độ chi tiết khác nhau.
- **Xử lý tín hiệu:**
 - Phân tích tín hiệu sinh học: EEG, ECG
 - Phân tích âm thanh: Nhận dạng giọng nói, nén âm thanh
 - Phân tích địa chấn: Phát hiện động đất
- **Phân tích dữ liệu:**
 - Phân tích dữ liệu tài chính
 - Phân tích dữ liệu khí tượng

2.3. Một số loại wavelet phổ biến

- **Haar wavelet:** Wavelet đơn giản nhất, có dạng hình chữ nhật.
- **Daubechies wavelet:** Một họ wavelet với các đặc tính khác nhau, được sử dụng rộng rãi trong xử lý ảnh.
- **Mexican hat wavelet:** Có dạng giống một chiếc mũ Mexico, thường được sử dụng để phát hiện các đặc trưng cục bộ.



2.4. Quy trình thực hiện biến đổi Wavelet

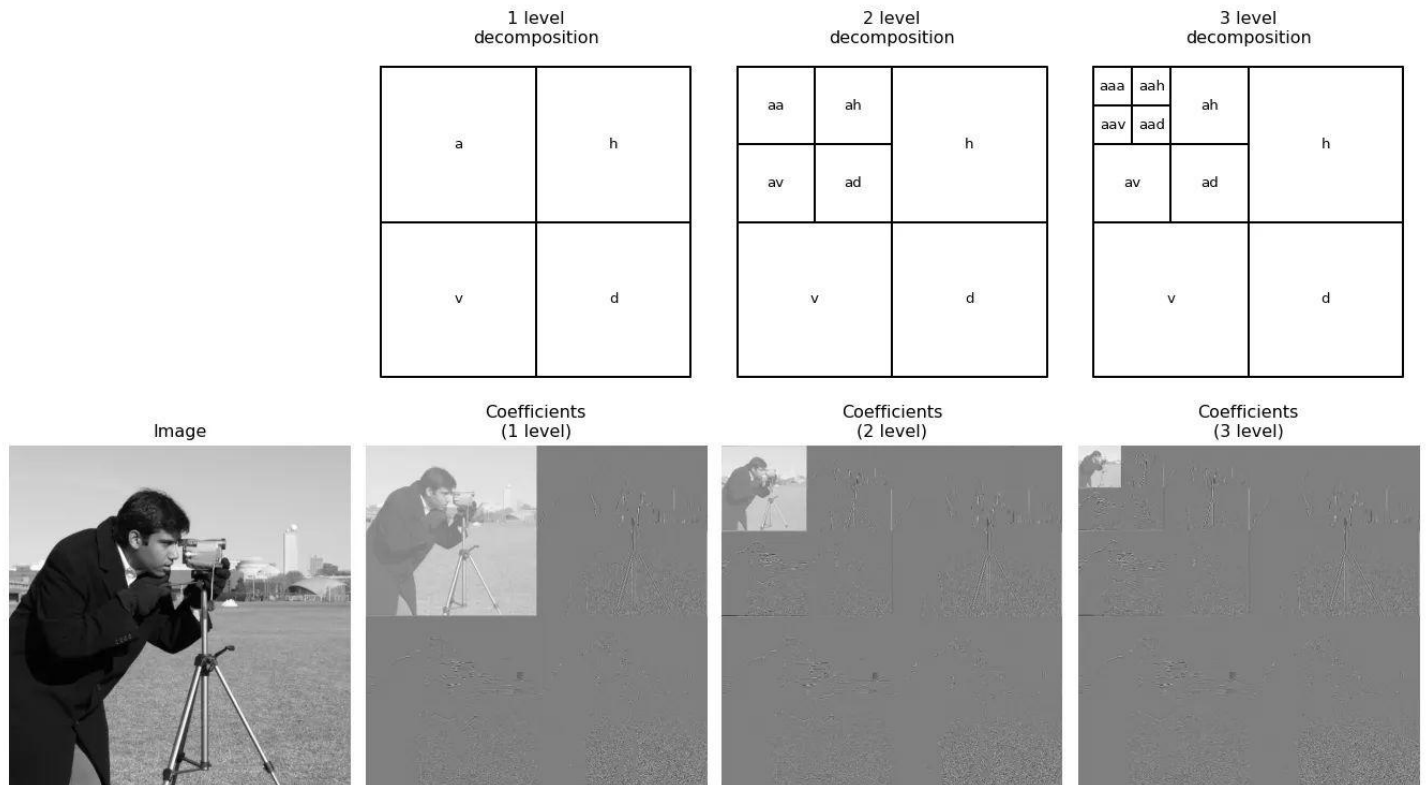
- **Chọn wavelet:** Lựa chọn loại wavelet phù hợp với loại tín hiệu cần phân tích.
- **Phân tích:** Phân tích tín hiệu thành các hệ số wavelet ở các mức độ chi tiết khác nhau.
- **Xử lý:** Thực hiện các phép biến đổi trên các hệ số wavelet (ví dụ: loại bỏ, tăng cường).
- **Tổng hợp:** Tổng hợp lại tín hiệu từ các hệ số wavelet đã được xử lý.

2.5. Biến đổi Wavelet rời rạc 2D (2D-DWT)

- **2D-DWT là một công cụ toán học phân tích hình ảnh thành nhiều băng tần**, cung cấp **phân tích đa độ phân giải**. Phân tích này cho phép phân tích, xử lý và nén hình ảnh hiệu quả.
 - Hãy tưởng tượng bạn đang xem một bức tranh sơn dầu trên Google Arts:
 - **Độ phân giải thấp (Zoom out):** Bạn nhìn thấy toàn bộ bức tranh, thấy bố cục, màu sắc chủ đạo, hình dáng nhân vật (nhưng không thấy nét cọ).
 - **Độ phân giải cao (Zoom in):** Bạn soi vào từng chi tiết nhỏ, thấy từng vết nứt trên tranh, từng sợi lông trên cây cọ (nhưng không thấy toàn cảnh).
 - → **2D-DWT giúp máy tính làm được cả hai việc này cùng lúc**. Nó tách bức ảnh ra thành nhiều lớp: một lớp chứa "toàn cảnh" và các lớp chứa "chi tiết nhỏ".
- **2D-DWT hoạt động như thế nào?**
 - **Phân tích hình ảnh:**
 - Hình ảnh được lọc bằng **bộ lọc thông thấp và thông cao** theo cả chiều ngang và chiều dọc.
 - Hình ảnh được **lọc thông thấp (Low Pass Filter)** (xấp xỉ) biểu diễn thông tin ở mức thô.
 - + Đây là cái sàng lỗ nhỏ. Nó giữ lại phần "tinh bột" mịn màng nhất. Trong ảnh, đây là **nền, mảng màu lớn, hình dáng chung**. Kết quả của nó là một phiên bản thu nhỏ và hơi mờ của ảnh gốc (gọi là ảnh Xấp xỉ - Approximation).
 - + *Ví dụ:* Nhìn khuôn mặt ai đó từ xa, bạn thấy da họ mịn màng.
 - Hình ảnh được **lọc thông cao (High Pass Filter)** (chi tiết) sẽ nắm bắt thông tin ở quy mô nhỏ theo hướng ngang, dọc và chéo.
 - + Đây là cái sàng lỗ to, giữ lại những cục đá sần sùi. Trong ảnh, đây là **biên giới, đường viền, vết sọc, hạt nhiễu**.
 - + *Ví dụ:* Lại gần khuôn mặt đó, bạn thấy nếp nhăn, lỗ chân lông.
 - + *Hướng:* Vì là 2D, nó bắt chi tiết theo 3 hướng: **Ngang** (mí mắt), **Dọc** (sống mũi), và **Chéo** (nếp nhăn khóe miệng).
 - **Phân tích lặp lại:**
 - Hình ảnh xấp xỉ có thể được phân tích thành các chi tiết thô và mịn hơn, tạo thành cây wavelet.
 - Mức độ phân tích quyết định mức độ chi tiết được ghi lại.

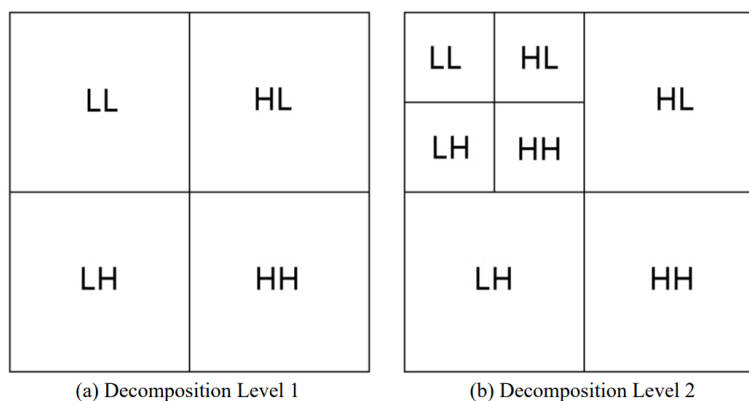
- **Các khái niệm chính:**

- **Bộ lọc Wavelet:** Việc lựa chọn bộ lọc wavelet ảnh hưởng đáng kể đến hiệu suất của 2D-DWT. Các bộ lọc phổ biến bao gồm Haar, Daubechies và Symlet
- **Cây Wavelet:** Cấu trúc phân cấp của hình ảnh phân tích, trong đó mỗi cấp độ biểu thị một độ phân giải khác nhau.
- **Hệ số wavelet:** Các hệ số thu được từ phép biến đổi wavelet biểu thị sự đóng góp của từng hàm cơ sở wavelet vào hình ảnh.

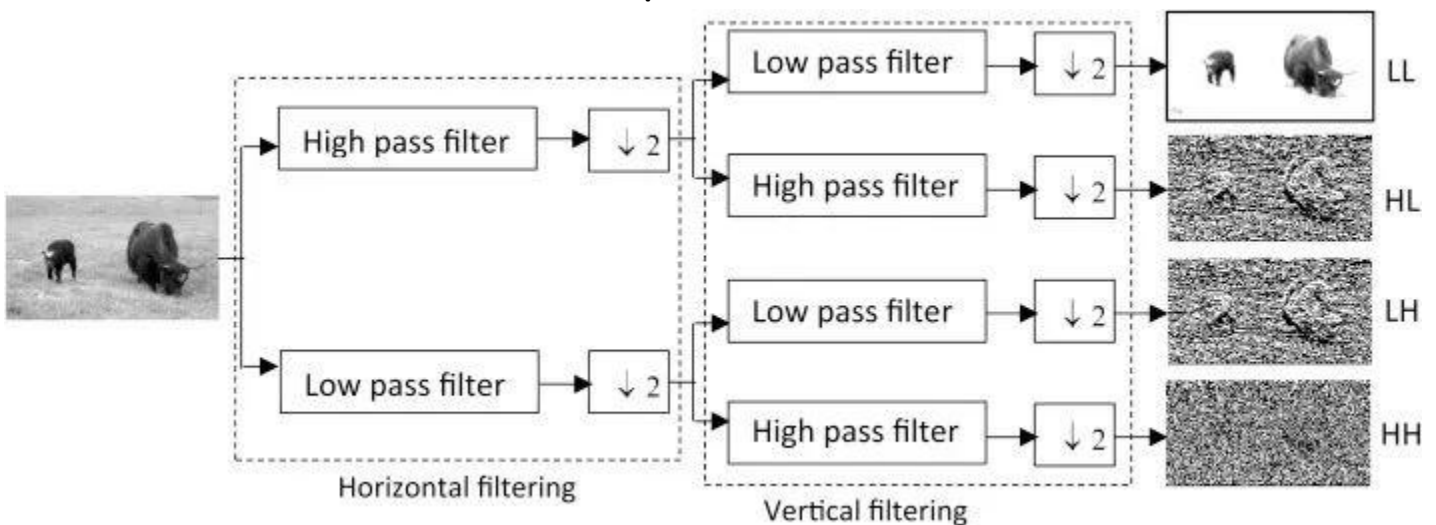


- Khi đưa ảnh qua 2D-DWT, ảnh được chia làm 4 phần nhỏ:

- **LL (Low-Low)/ a (approximate):** Ảnh xấp xỉ, thu nhỏ, chứa thông tin tần số thấp (nhìn giống ảnh gốc nhưng mờ/nhỏ hơn).
- **LH (Low-High)/ h (horizontal):** Chứa chi tiết cạnh ngang.
- **HL (High-Low)/ v (vertical):** Chứa chi tiết cạnh dọc.
- **HH (High-High)/ d (diagonal, đường chéo chính):** Chứa chi tiết cạnh chéo (thường là nhiễu).

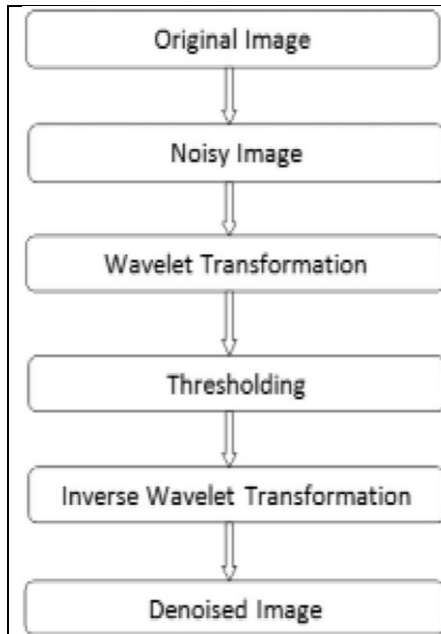


- **h, v, d** là các chi tiết biên (**tần số cao**) → Chính **h, v, d** (Detail): Lưu giữ **chi tiết ở từng hướng** → muốn **bỏ bớt nhiều** hoặc chi tiết nhỏ.
- Quá trình này có thể lặp lại trên ô LL để phân tích sâu hơn (Level 2, Level 3...).
 - **Level 1:**
 - Ảnh gốc được tách thành 4 sub-bands: a, h, v, d.
 - **a giữ thông tin chính** (giống ảnh gốc đã được làm mờ).
 - **h, v, d** là các thành phần chi tiết (biên, góc, hoa văn...).
 - **Level 2:**
 - **Lấy tiếp phần a ở cấp 1** để phân tích tiếp, thu được a, h, v, d mới.
 - Tương tự, **h, v, d ở cấp 1** giữ nguyên không phân rã tiếp.
 - Kết quả: Thêm 4 sub-bands mới từ phần a cấp 1.
 - **Level 3** (và cao hơn):
 - Tiếp tục phân rã phần a của cấp 2, tạo thêm các sub-bands a, h, v, d mới.
 - Quá trình lặp lại cho đến khi đạt số cấp mong muốn hoặc kích thước ảnh quá nhỏ.
 - **Mỗi cấp độ càng cao thì ảnh "a" càng mờ** (chứa thông tin tần số rất thấp), còn "h, v, d" chứa chi tiết ở những tần số khác nhau.
 - Ở cấp phân rã thấp (ví dụ cấp 1), phần a vẫn giữ được nhiều chi tiết của ảnh, giúp duy trì độ nét → **Tuy nhiên, các hệ số h, v, d ở cấp này có thể chứa nhiễu đáng kể.**
Vì muốn tăng level "a" để giảm nhiễu ⇒ có thể dùng "Threshold" ở mỗi level
 - Cấu trúc này giống như rẽ cây hay kim tự tháp ngược, nên gọi là **"Cây Wavelet"**. Càng đi sâu, bức ảnh càng nhỏ và càng cô đọng.
- "Nếu muốn làm mờ bức ảnh (xóa nếp nhăn, làm mịn da), bạn sẽ can thiệp vào thành phần nào?" Bạn sẽ **giảm bớt hoặc loại bỏ** các thành phần **HL, LH, HH** (vì đây là nơi chứa nếp nhăn, gai góc - tần số cao).
- **Phân tích hình ảnh biến đổi wavelet rời rạc 2 chiều**



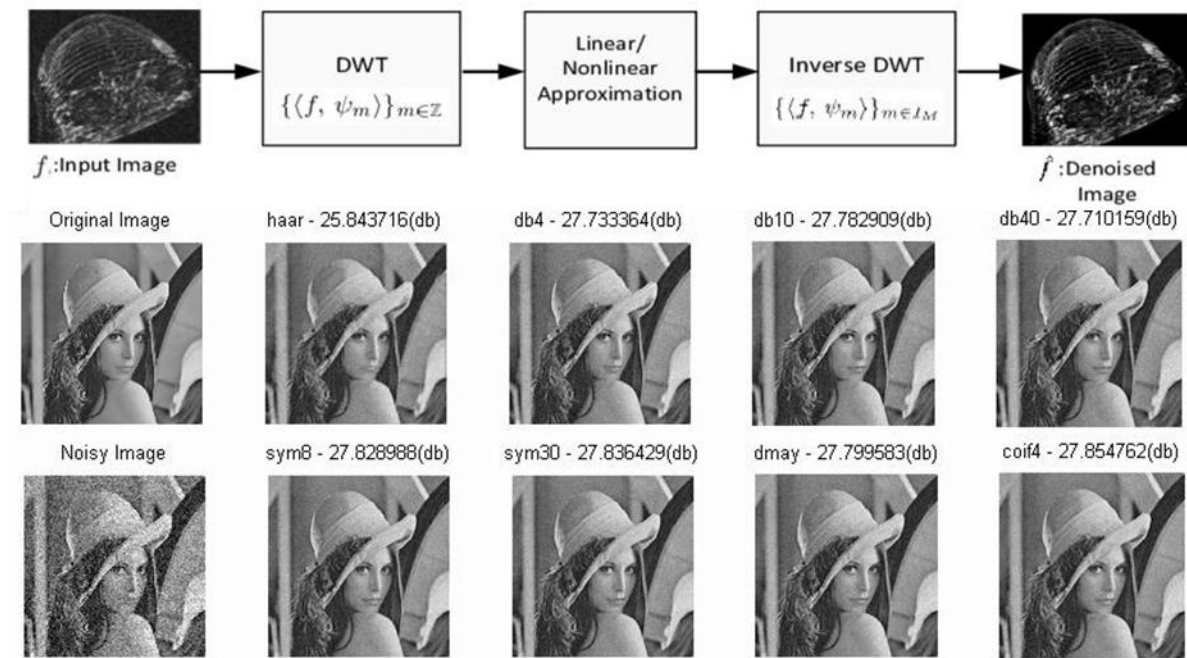
- Xử lý ảnh là 2D (chiều ngang và chiều dọc), nên máy tính phải làm lần lượt:
 - **Bước 1: Lọc theo chiều Ngang (Horizontal filtering - Khung nét đứt bên trái):** Máy tính quét từng hàng ngang của bức ảnh:
 - Tách ảnh thành 2 phần: Một phần chứa chi tiết ngang mượt mà (L), một phần chứa chi tiết ngang thay đổi gắt (H).
 - **Ký hiệu $\downarrow 2$:** Nghĩa là "Giảm mẫu" (Downsampling). Vì đã lọc xong, ta vớt bớt một nửa dữ liệu để giảm dung lượng (ví dụ: cứ 2 điểm ảnh thì giữ lại 1).
 - **Bước 2: Lọc theo chiều Dọc (Vertical filtering - Khung nét đứt bên phải):** Từ kết quả của bước 1, máy tính tiếp tục quét theo chiều dọc (cột): Nó lại lấy từng phần (L và H cũ) ra để lọc tiếp một lần nữa bằng 2 cái sàng (Low và High).
- Kết quả cuối cùng là 4 bức ảnh con ở bên phải, tương ứng với 4 tổ hợp sàng lọc:
 - **LL (Low-Low) - Ảnh trên cùng:**
 - *Quy trình:* Qua sàng Mịn (Ngang) $\rightarrow$ rồi qua tiếp sàng Mịn (Dọc).
 - *Kết quả:* Đây là **bản thu nhỏ của ảnh gốc**. Nó giữ lại toàn bộ thông tin quan trọng nhất nhưng mờ hơn và nhỏ hơn. Máy tính sẽ dùng cái này nếu cần nhận diện vật thể tổng quát (ví dụ: đây là con trâu).
 - **HL và LH - Hai ảnh giữa:**
 - Đây là sự kết hợp của 1 lần sàng Mịn và 1 lần sàng Gai góc.
 - Chúng có nhiệm vụ **bắt các cạnh theo hướng cụ thể**:
 - Một cái sẽ làm nổi bật các **đường kẻ dọc** (thân cây, chân trâu).
 - Một cái sẽ làm nổi bật các **đường kẻ ngang** (đường chân trời, lưng trâu).
 - **HH (High-High) - Ảnh dưới cùng:**
 - *Quy trình:* Qua sàng Gai góc (Ngang) $\rightarrow$ rồi qua tiếp sàng Gai góc (Dọc).
 - *Kết quả:* Nó bắt các **đường chéo** và các **điểm nhiễu** (các chấm lấm tấm). Đây là nơi chứa những chi tiết vụn vặt nhất của bức ảnh.
- **Ý nghĩa thực tế:** Tại sao người ta lại làm trò phức tạp này? Để **nén ảnh** (như JPEG 2000). **Khi nén, máy tính sẽ giữ lại nguyên vẹn ô LL (ảnh thu nhỏ) vì nó quan trọng nhất. Còn với 3 ô đen xì còn lại (HL, LH, HH), máy tính biết đó chỉ là đường viền và nhiễu, nó có thể xóa bớt hoặc nén rất mạnh tay mà mắt người không nhận ra sự khác biệt lớn.**

• Wavelet based Denoising of Images



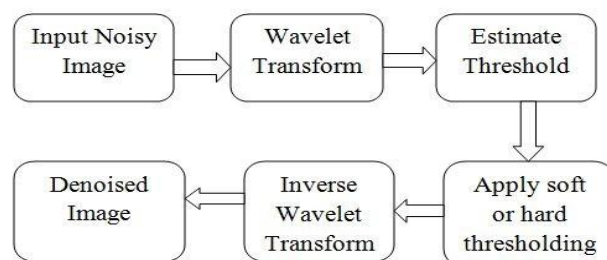
Hãy tưởng tượng bạn có một chiếc áo dính đầy bụi (Ảnh nhiễu).

1. **Original Image & Noisy Image:** Bạn có ảnh gốc, sau đó bị thêm nhiễu (giống như tivi bị nhiễu hạt mè, hoặc ảnh chụp thiếu sáng bị sạn - xem hình Noisy Image bên phải).
2. **Wavelet Transformation (Phân rã):** Đây là bước bạn tháo rời chiếc áo ra thành từng sợi vải: sợi thô (LL) và các sợi mảnh/bụi bám (HL, LH, HH).
3. **Thresholding (Phân ngưỡng):** Đây chính là bước "vỗ bụi" - **Bước quan trọng nhất**.
 - **Vấn đề:** Trong các ô chi tiết (HL, LH, HH), có cả **nét vẽ thật** (cạnh của vật thể) và **hạt nhiễu**. Làm sao máy tính phân biệt được? → *Giải pháp:* Dựa vào độ lớn (Biên độ).
 - **Nét vẽ thật (Cạnh):** Thường tạo ra hệ số rất lớn (sự thay đổi màu sắc mạnh).
 - **Hạt nhiễu:** Thường chỉ tạo ra các hệ số nhỏ liti (sự thay đổi màu sắc yếu).
 - **Hành động:** Bạn đặt ra một mức sàn (Threshold).
 - *Anh nào bé hơn mức sàn này → Là bụi → Xóa về 0.*
 - *Anh nào lớn hơn mức sàn → Là nét vẽ → Giữ lại.*
4. **Inverse Wavelet Transformation (Biến đổi ngược):** May lại chiếc áo từ những sợi vải đã được làm sạch.
5. **Denoised Image:** Bạn nhận được bức ảnh sạch đẹp (xem các ảnh kết quả ở hàng dưới bên phải).



- Tại sao lại có nhiều hình "Lena"? Bạn nhìn thấy các chữ như haar, db4, sym8, coif4... kèm theo các con số (db).
- Các loại "chổi quét" khác nhau: Wavelet không chỉ có một loại.
 - **Haar**: Là loại đơn giản nhất (vuông vức).
 - **Daubechies (db)**, **Symlet (sym)**, **Coiflet (coif)**: Là các loại phức tạp hơn, uốn lượn mềm mại hơn.
- Ý nghĩa: Tùy vào bức ảnh của bạn có nhiều đường thẳng hay đường cong mà bạn chọn loại "chổi" (hàm Wavelet) phù hợp để quét sạch bụi nhất mà không làm xước ảnh.
- Con số (db - Decibel): Chỉ số **PSNR** (Tỷ lệ tín hiệu trên nhiễu). **Số càng to nghĩa là ảnh càng sạch và càng giống ảnh gốc**. Sau khi khử nhiễu, các chỉ số này đều cao hơn hẳn so với ảnh nhiễu.

• Denoising using hard-thresholding



- Tại sao phải có bước **Ước lượng ngưỡng** (Estimate Threshold) mà không chọn bừa một số?
 - Mỗi bức ảnh có mức độ nhiễu khác nhau (ảnh chụp đêm nhiễu nhiều, ảnh chụp ngày nhiễu ít).
 - **Bước này dùng toán học (thường là thống kê) để tính toán** xem: "Với bức ảnh cụ thể này, mức độ dao động của nhiễu là bao nhiêu?". Từ đó, nó tự động đề xuất một con số ngưỡng an toàn (ví dụ: ngưỡng VisuShrink, BayesShrink...).
 - **Giúp tránh tình trạng chọn ngưỡng quá cao (mất chi tiết) hoặc quá thấp (còn nguyên nhiễu).**

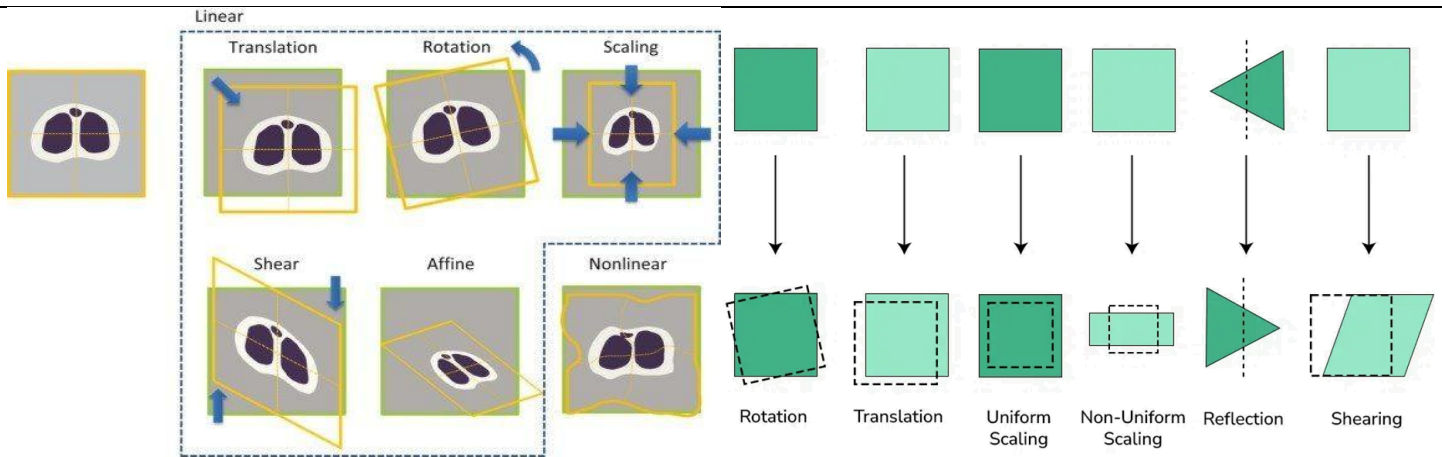
- **Apply Soft or Hard Thresholding (Áp dụng ngưỡng Mềm hoặc Cứng):** Đây là hai chiến thuật "xử lý" các hệ số khác nhau:
 - **Hard Thresholding (Ngưỡng Cứng):**
 - Quy tắc: "Anh nào dưới chuẩn → Chém bỏ (về 0). Anh nào trên chuẩn → Giữ nguyên."
 - **Hậu quả:** Rất dứt khoát, nhưng đôi khi tạo ra những vết gãy khúc, gián đoạn trong ảnh (vì đang giữ nguyên bỗng nhiên tụt xuống 0).
 - **Soft Thresholding (Ngưỡng Mềm):**
 - Quy tắc: "Anh nào dưới chuẩn → Chém bỏ. Anh nào trên chuẩn → Thu nhỏ lại một chút (trừ đi giá trị ngưỡng)."
 - **Hậu quả:** Nó làm cho sự chuyển tiếp giữa phần bị xóa và phần giữ lại trở nên mượt mà hơn.
 - → Trong xử lý ảnh, người ta thường thích dùng Soft Thresholding hơn vì nó giúp ảnh sau khi khử nhiễu nhìn tự nhiên hơn, ít bị các vết gợn sóng nhân tạo (artifacts).

3. Biến đổi hình học trong xử lý ảnh (Geometric Transformations)

- **Biến đổi hình học** là một kỹ thuật cơ bản và quan trọng trong xử lý ảnh, cho phép chúng ta thay đổi hình dạng, kích thước hoặc vị trí của một hình ảnh bằng cách thao tác với các pixel của nó dựa trên các phép toán học. Những phép biến đổi này được sử dụng để sửa đổi vị trí, kích thước, xoay hoặc hình dạng của hình ảnh.
- Điều này rất hữu ích trong nhiều ứng dụng như:
 - **Hiệu chỉnh ảnh:** Sửa các lỗi biến dạng do góc chụp, ống kính gây ra.
 - Khi chụp ảnh tòa nhà cao tầng từ dưới lên, tòa nhà trông như bị đổ nghiêng. Ta dùng biến đổi hình học để "nắn" nó thẳng lại.
 - **Nhận dạng vật thể:** Căn chỉnh các đối tượng để dễ dàng so sánh và phân loại.
 - Để so sánh khuôn mặt A và khuôn mặt B, máy tính cần xoay cho cả hai khuôn mặt cùng nhìn thẳng thì mới so sánh chính xác được.
 - **Tạo hiệu ứng đặc biệt:** Phóng to, thu nhỏ, xoay, biến dạng ảnh để tạo ra các hiệu ứng nghệ thuật hoặc phục vụ cho các mục đích khác.
 - **Khâu tiền xử lý:** Chuẩn bị ảnh trước khi đưa vào các thuật toán xử lý ảnh khác như nhận dạng khuôn mặt, phân đoạn ảnh.
 - Trước khi đưa ảnh vào AI, tất cả ảnh phải được resize về cùng một kích thước (ví dụ 256x256).

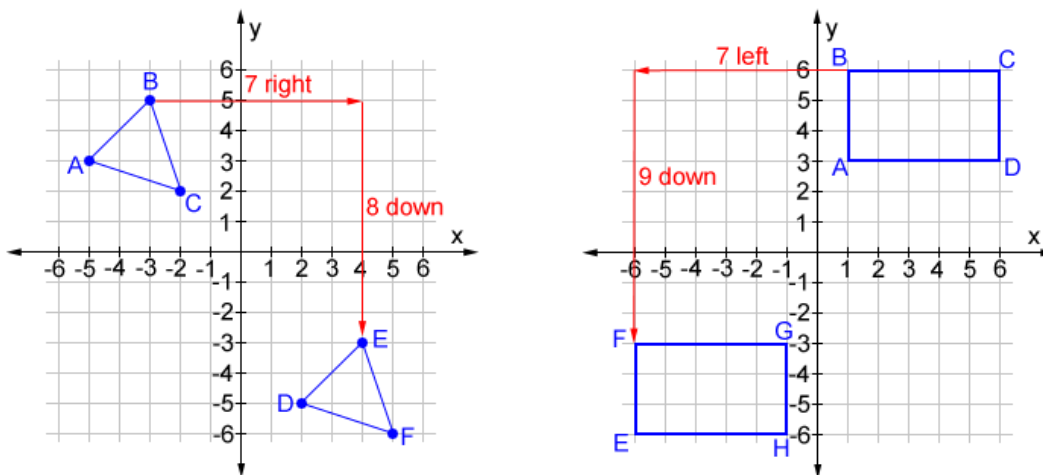
3.1. Các loại biến đổi hình học phổ biến

- **Phép tịnh tiến:** Di chuyển toàn bộ hình ảnh theo một vectơ cho trước.
- **Phép quay:** Xoay hình ảnh quanh một điểm cố định một góc nhất định.
- **Phép tỉ lệ:** Phóng to hoặc thu nhỏ hình ảnh theo một tỷ lệ nhất định.
- **Phép biến dạng:** Thay đổi hình dạng của hình ảnh theo các hướng khác nhau.
- **Phép chiếu:** Chiếu một hình ảnh 3D lên một mặt phẳng 2D.



• Translation

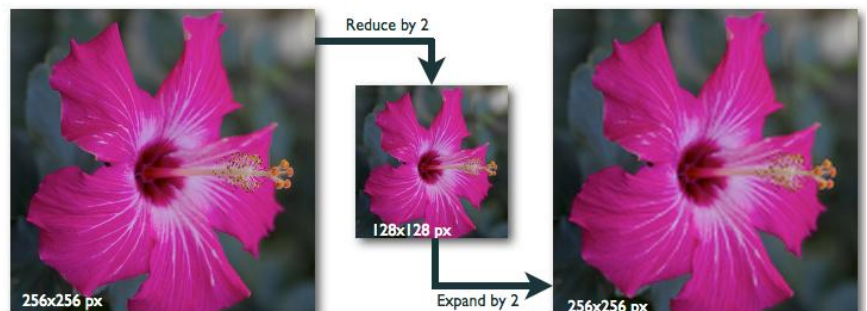
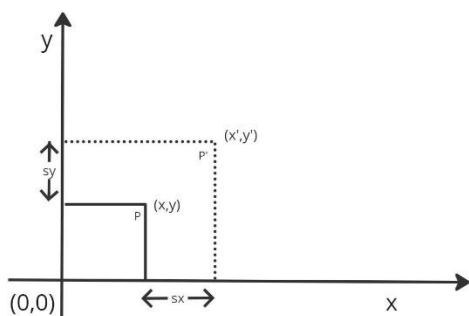
- Shifting an image in **a specific direction** (horizontally and/or vertically).
- This transformation moves every pixel by a constant offset (Biến đổi này di chuyển mọi pixel theo độ lệch không đổi).



• Scaling

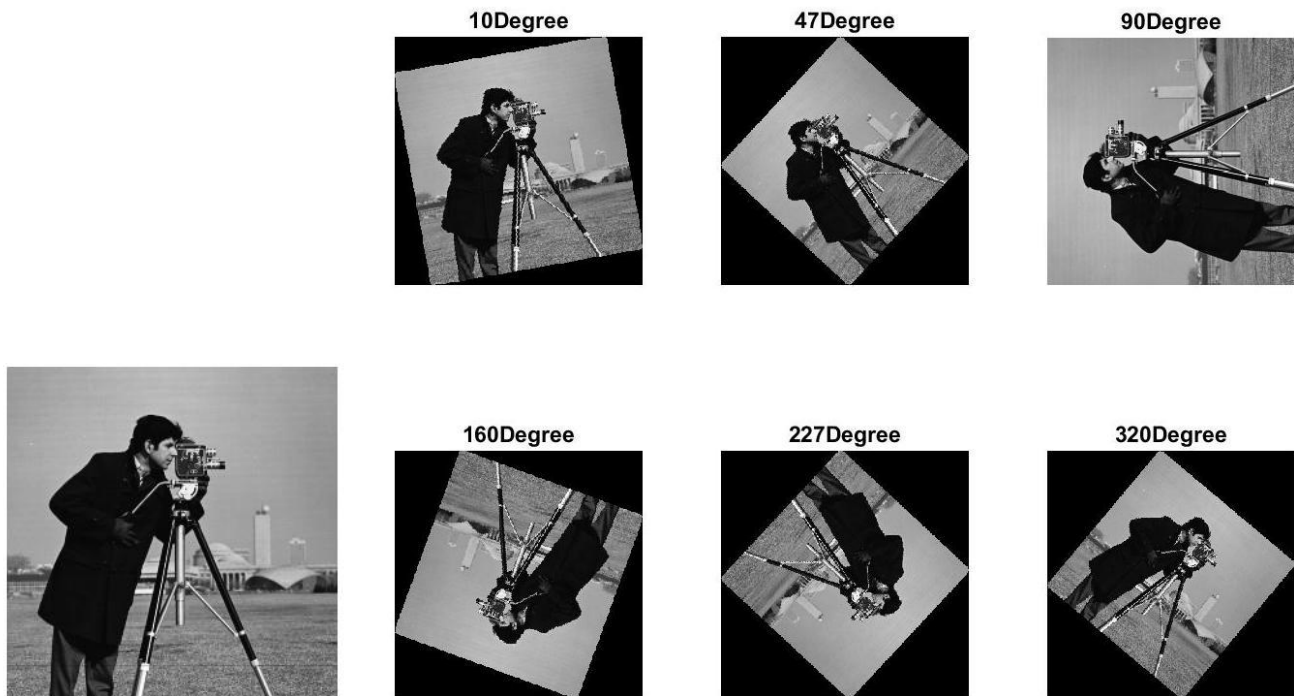
- Changing the size of an image (enlarging or reducing).
- This involves multiplying pixel coordinates by scaling factors (nhân tọa độ pixel với các hệ số tỷ lệ).

2D Scaling



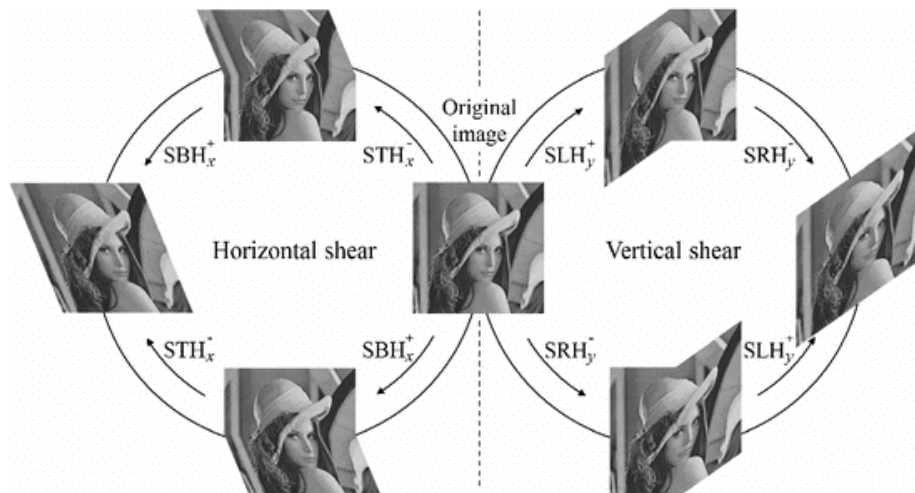
- **Rotation**

- Turning an image around a specific point.
- This transformation involves rotating pixel coordinates by a certain angle (xoay tọa độ pixel theo một góc nhất định).



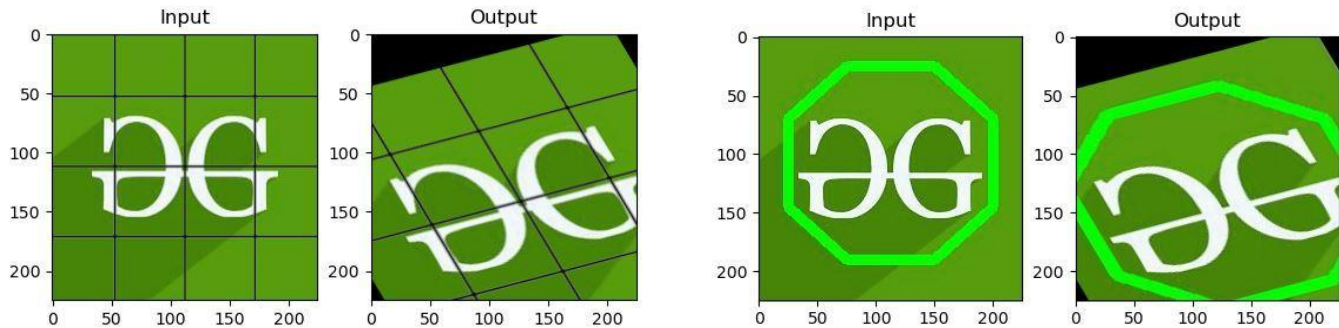
- **Shearing (Skewing)**

- Distorting an image by shifting pixels along one axis (Biến dạng hình ảnh bằng cách dịch chuyển các pixel dọc theo một trục).
- This transformation skews the image (làm sai lệch hình ảnh).



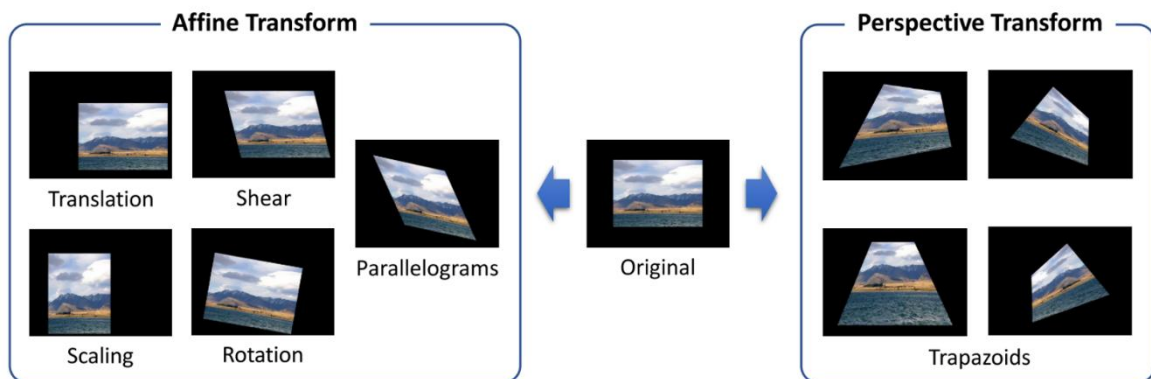
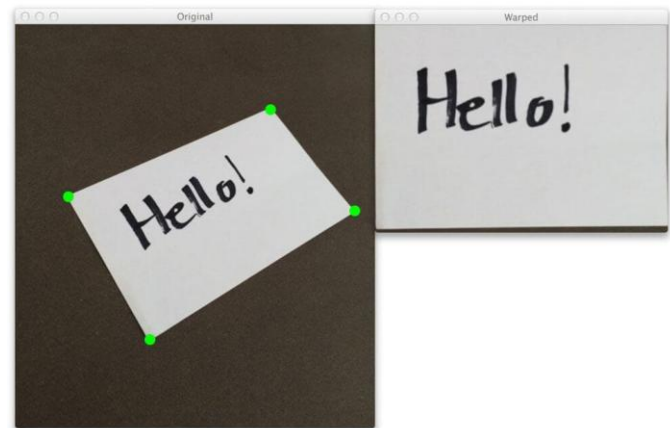
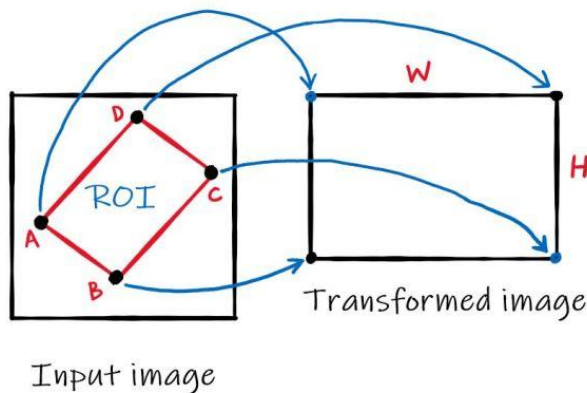
Affine Transformation

- A combination of translation, scaling, rotation, and shearing.
- Hãy tưởng tượng bạn ép nghiêng hình vuông thành **hình bình hành**.
- *Đặc điểm quan trọng:* Các đường thẳng song song vẫn giữ nguyên là **song song** → **Preserves parallel lines** (**Bảo toàn các đường song song**) (Ví dụ: hai cạnh đối diện của hình bình hành vẫn song song).



Perspective Transformation (Chuyển đổi phối cảnh)

- More complex transformations that can change the perspective of an image.
- Used for correcting perspective distortions (Được sử dụng để chỉnh biến dạng phối cảnh).

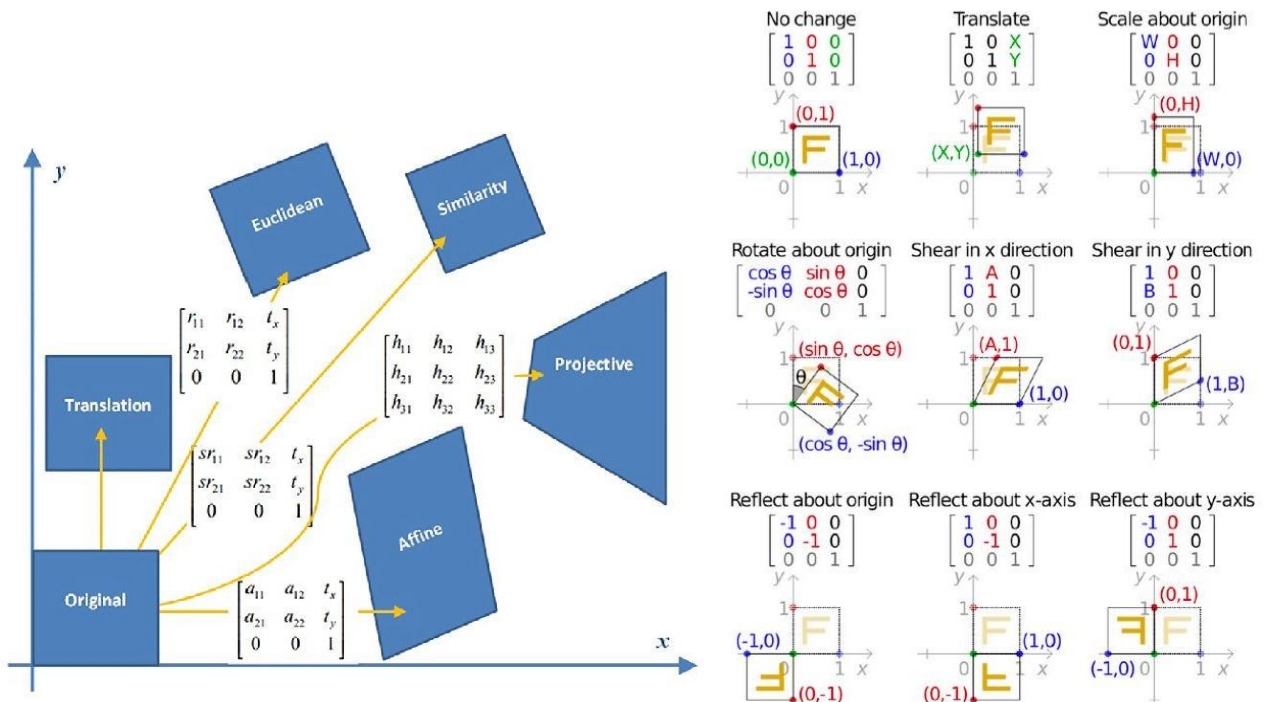


$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ 0 & 0 & 1 \end{bmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \quad \begin{matrix} \text{2x3 matrix} \\ \text{(6 DOF)} \end{matrix}$$

$$\begin{pmatrix} wx' \\ wy' \\ w \end{pmatrix} = \begin{bmatrix} p_{11} & p_{12} & p_{13} \\ p_{21} & p_{22} & p_{23} \\ p_{31} & p_{32} & 1 \end{bmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \quad \begin{matrix} \text{3x3 matrix} \\ \text{(8 DOF)} \end{matrix}$$

3.2. Ma trận biến đổi

- Để mô tả các phép biến đổi hình học, chúng ta thường sử dụng **ma trận biến đổi**. **Ma trận này sẽ nhân với tọa độ của mỗi điểm ảnh để tính toán tọa độ mới sau khi biến đổi.**
 - Ma trận tịnh tiến:** Thường được biểu diễn dưới dạng một ma trận đồng nhất (homogeneous matrix) để bao gồm cả phép tịnh tiến và các phép biến đổi khác.
 - Ma trận quay:** Được xây dựng dựa trên các hàm lượng giác **sin** và **cos**.
 - Ma trận tỉ lệ:** Là một ma trận đường chéo với các phần tử trên đường chéo chính là các hệ số tỉ lệ.



3.3. Phương pháp Nội suy

- Nội suy là gì? (Hiểu đơn giản):** Hãy tưởng tượng bạn có một tấm lưới co giãn (bức ảnh). Khi bạn **phóng to, xoay, hoặc làm méo** tấm lưới đó, các mắt lưới cũ (pixel gốc) sẽ bị xô dịch và không còn nằm ngay ngắn trên các vị trí hàng/cột chuẩn của màn hình nữa.
 - Lúc này, màn hình máy tính sẽ có những "chỗ trống" mới cần được tô màu. Máy tính sẽ tự hỏi: *"Tại cái điểm mới này, tôi nên tô màu gì đây?"*
 - **Nội suy** chính là cách máy tính nhìn vào các điểm ảnh cũ xung quanh để **tính toán ("đoán") ra màu sắc cho điểm ảnh mới**.
- Khi biến đổi hình ảnh, **các điểm ảnh mới thường không trùng khớp với các điểm ảnh gốc**. Do đó, ta cần sử dụng các **phương pháp nội suy** để tính toán giá trị màu tại các điểm ảnh mới.
- Một số phương pháp nội suy phổ biến:**
 - Nearest Neighbor:** Gán cho điểm ảnh mới giá trị của điểm ảnh gần nhất trong ảnh gốc.
 - "**Thằng hàng xóm nào gần mình nhất thì mình bắt chước y hệt màu của nó**".
 - Kết quả:** Ảnh sẽ **rất sắc cạnh** nhưng bị **răng cưa** (hiệu ứng khối vuông).
 - Ưu điểm:** Tốc độ nhanh nhất.

- **Bilinear:** Nội suy tuyến tính dựa trên giá trị của bốn điểm ảnh xung quanh.
 - "Mình sẽ lấy màu của **4 thẳng hàng xóm** xung quanh rồi pha trộn trung bình lại".
 - **Kết quả:** Ảnh mượt hơn, hết răng cưa, nhưng trông sẽ hơi mờ (nhòe).
- **Bicubic:** Nội suy bậc ba, cho kết quả mịn hơn nhưng phức tạp hơn.
 - "Mình sẽ xem xét kỹ lưỡng tới **16 thẳng hàng xóm** (lấy rộng hơn) và dùng hàm toán học phức tạp (bậc ba) để tính toán sự thay đổi màu sắc".
 - **Kết quả:** Ảnh vừa mượt mà, vừa giữ được độ sắc nét tốt hơn Bilinear. Đây là chuẩn mực cho việc chỉnh sửa ảnh chụp.

3.4. Biến đổi hình học trong xử lý ảnh với thư viện OpenCV và Pillow

- **Ứng dụng trong OpenCV:** OpenCV là một thư viện xử lý ảnh phổ biến cung cấp các hàm để thực hiện các phép biến đổi hình học một cách dễ dàng. Các hàm này thường nhận vào ma trận biến đổi và hình ảnh đầu vào, trả về hình ảnh sau khi biến đổi.
- **OpenCV và Pillow** là hai thư viện phổ biến trong Python được sử dụng để thực hiện các biến đổi hình học. Mỗi thư viện có những ưu điểm và cách sử dụng khác nhau.
- **Thay đổi kích thước:**
 - **Thu nhỏ:** Giảm số lượng pixel của hình ảnh.
 - **Phóng to:** Tăng số lượng pixel của hình ảnh.
 - **OpenCV:** cv2.resize()
 - **Pillow:** img.resize()
- **Xoay:** Xoay hình ảnh quanh một điểm trung tâm.
 - **OpenCV:** cv2.rotate(), cv2.warpAffine()
 - **Pillow:** img.rotate()
- **Cắt xén:** Lấy một phần của hình ảnh.
 - **OpenCV:** img[y:y+h, x:x+w]
 - **Pillow:** img.crop()
- **Biến đổi affine:** Bao gồm các phép biến đổi cơ bản như tịnh tiến, xoay, tỉ lệ.
 - **OpenCV:** cv2.warpAffine()
- **Biến đổi phối cảnh:** Biến đổi hình ảnh 3D thành 2D.
 - **OpenCV:** cv2.perspectiveTransform()

```
import cv2
import numpy as np

# Đọc ảnh
img = cv2.imread('image.jpg')

# Thay đổi kích thước
resized_img = cv2.resize(img, (200, 100))

# Xoay ảnh 45 độ
rows, cols = img.shape[:2]
M = cv2.getRotationMatrix2D((cols/2, rows/2), 45, 1)
rotated_img = cv2.warpAffine(img, M, (cols, rows))

# Hiển thị ảnh
cv2.imshow('Original', img)
cv2.imshow('Resized', resized_img)
cv2.imshow('Rotated', rotated_img)
cv2.waitKey(0)
```

```
from PIL import Image

# Đọc ảnh
img = Image.open('image.jpg')

# Thay đổi kích thước
resized_img = img.resize((200, 100))

# Xoay ảnh 45 độ
rotated_img = img.rotate(45)

# Hiển thị ảnh
resized_img.show()
rotated_img.show()
```

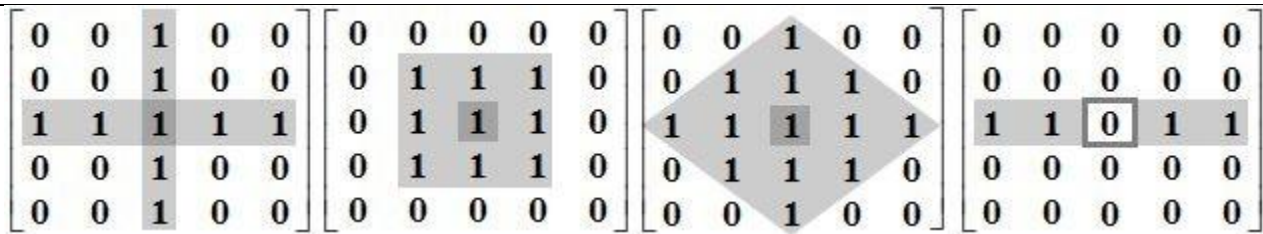
4. Xử lý hình thái học (Morphology)

- Thực chất đây là kỹ thuật "**Chỉnh sửa hình dạng**".
- Là một **tập hợp các phép toán dựa trên hình học**, được sử dụng để **xử lý các đối tượng trong ảnh nhị phân hoặc ảnh xám**.
- Các phép toán này thường được sử dụng để *đơn giản hóa hình dạng của đối tượng, loại bỏ nhiễu, hoặc trích xuất các đặc trưng hình học*.
- **Các phép toán cơ bản trong xử lý hình thái học**
 - **Phép giãn nở (Dilation)**: Làm **tăng kích thước** của các đối tượng trong ảnh.
 - **Hành động**: Đắp thêm điểm ảnh vào rìa đối tượng.
 - **Kết quả**: Vật thể phình to ra, béo lên.
 - **Công dụng**: Dùng để **lấp các lỗ thủng nhỏ** bên trong vật thể hoặc nối liền các nét đứt đoạn.
 - **Phép xói mòn (Erosion)**: Làm **giảm kích thước** của các đối tượng trong ảnh.
 - **Hành động**: Gọt bớt các điểm ảnh ở rìa đối tượng.
 - **Kết quả**: Vật thể bị co nhỏ lại, gầy đi.
 - **Công dụng**: Dùng để **xóa các vết nhiễu nhỏ** (các chấm trắng li ti). Nếu chấm trắng nhỏ hơn cái "khuôn", nó sẽ bị gọt mất tích luôn.
 - **Phép mở (Opening)**: **Kết hợp phép xói mòn và giãn nở**, thường được dùng để loại bỏ các đối tượng nhỏ và làm mịn đường viền.
 - **Xói mòn trước → Giãn nở sau**.
 - *Tại sao?* Đầu tiên dùng "Xói mòn" để giết chết các hạt nhiễu nhỏ. Sau đó dùng "Giãn nở" để hồi phục lại kích thước của vật thể chính (nhưng hạt nhiễu đã chết rồi thì không hồi lại được).
 - **Dùng khi**: Muốn xóa nhiễu "muối tiêu" (các chấm trắng lấm tấm trên nền đen).

- **Phép đóng (Closing):** **Kết hợp phép giãn nở và xói mòn**, thường được dùng để lấp đầy các lỗ hổng nhỏ bên trong các đối tượng.
 - **Giãn nở trước → Xói mòn sau.**
 - **Tại sao?** Đầu tiên dùng "Giãn nở" để lấp đầy các khe nứt, lỗ thủng. Sau đó dùng "Xói mòn" để gọt bớt phần thừa cho về lại kích thước cũ.
 - **Dùng khi:** Muốn làm liền mạch vật thể, lấp lỗ thủng.
- Ngoài các phép toán cơ bản, còn có các phép toán hình thái học khác như:
 - **Gradient hình thái học:** Tính toán độ dốc của hình dạng.
 - **Top-hat:** Tìm các đối tượng sáng hoặc tối nhỏ.
 - **Black-hat:** Tìm các lỗ hổng nhỏ.
- **Ứng dụng của xử lý hình thái học**
 - **Loại bỏ nhiễu:** Loại bỏ các điểm nhiễu nhỏ bằng phép mở.
 - **Làm mịn đường viền:** Làm mịn các đường viền răng cưa bằng phép mở và đóng.
 - **Phân tách các đối tượng dính nhau:** Sử dụng phép xói mòn để tách các đối tượng dính nhau.
 - **Lấp đầy các lỗ hổng:** Sử dụng phép đóng để lấp đầy các lỗ hổng nhỏ bên trong các đối tượng.
 - **Trích xuất đặc trưng:** Tính toán các đặc trưng hình học như diện tích, chu vi, hình dạng.

• **Phần tử cấu trúc (Structuring element)**

- **Phần tử cấu trúc ảnh (Image structuring element)** là một hình khối được định nghĩa sẵn nhằm tương tác với ảnh xem nó có thỏa mãn một số tính chất nào đó.
- Để chỉnh sửa hình dạng, bạn cần một cái "khuôn" hoặc một cái "chổi". Trong thuật toán, người ta gọi là **Phần tử cấu trúc**.
- Nó chỉ là một ma trận nhỏ (như hình minh họa các ô số 0 và 1 trong slide).
- **Hình dáng:** Có thể là hình vuông, hình tròn, hoặc hình chữ thập...
- **Tác dụng:** Máy tính sẽ cầm cái "khuôn" này rà đi rà lại khắp bức ảnh để xem chỗ nào vừa khít, chỗ nào thừa, chỗ nào thiếu.
- **Các cấu trúc phần tử phổ biến gồm:**
 - **Hình chữ nhật:** Dùng để làm mịn các cạnh theo hướng ngang hoặc dọc.
 - **Hình tròn:** Dùng để làm mịn các cạnh theo mọi hướng.
 - **Hình chữ thập:** Dùng để loại bỏ các điểm đơn lẻ.
- **Các thành phần với một phần tử cấu trúc:**
 1. **Kích thước** của ma trận phần tử cấu trúc.
 2. **Hình dáng** của phần tử cấu trúc.
 3. **Gốc** của phần tử (Ogiri). **Thông thường thì gốc của phần tử mang giá trị 1 nhưng trong một số trường hợp thì gốc phần tử mang giá trị C** (Phần tử cấu trúc không có gốc).



Phép biến đổi	Tác dụng chính	Cấu trúc phần tử	Ứng dụng điển hình
Xói mòn (Erosion)	Thu nhỏ đối tượng, loại bỏ các điểm lẻ tẻ, làm mịn đường viền bên trong	Bất kỳ	Loại bỏ nhiễu, tách các đối tượng dính nhau
Giãn nở (Dilation)	Phóng to đối tượng, lấp đầy các lỗ hổng nhỏ, làm dày đường viền	Bất kỳ	Lấp đầy các lỗ hổng, kết nối các đối tượng gần nhau
Mở (Opening)	Loại bỏ các đối tượng nhỏ, làm mịn đường viền bên ngoài	Hình chữ nhật, hình tròn	Loại bỏ nhiễu, tách các đối tượng dính nhau nhẹ
Đóng (Closing)	Lấp đầy các lỗ hổng nhỏ, làm mịn đường viền bên trong	Hình chữ nhật, hình tròn	Lấp đầy các lỗ hổng, kết nối các đối tượng gần nhau
Gradient hình thái học	Tính toán độ dốc của hình dạng, tìm các biên	Bất kỳ	Phát hiện biên, phân tích hình dạng
Top-hat	Tìm các đối tượng sáng nhỏ trên nền sáng	Hình chữ nhật, hình tròn	Phát hiện các đối tượng nhỏ, phân tích kết cấu
Black-hat	Tìm các lỗ hổng nhỏ trên nền tối	Hình chữ nhật, hình tròn	Phát hiện các lỗ hổng nhỏ, phân tích kết cấu

• Các phép biến đổi hình thái cơ bản trong OpenCV

- **Erosion (Xói mòn):** Làm thu nhỏ các vùng sáng trong ảnh, loại bỏ các điểm nhiễu nhỏ, làm mịn đường viền bên trong → `cv2.erode()`
- **Dilation (Giãn nở):** Làm tăng kích thước các vùng sáng, lấp đầy các lỗ hổng nhỏ, làm dày đường viền → `cv2.dilate()`
- **Opening (Mở):** Kết hợp erosion và dilation, thường dùng để loại bỏ nhiễu và làm mịn đường viền bên ngoài → `cv2.morphologyEx(img, cv2.MORPH_OPEN, kernel)`
- **Closing (Đóng):** Kết hợp dilation và erosion, thường dùng để lấp đầy các lỗ hổng nhỏ và làm mịn đường viền bên trong → `cv2.morphologyEx(img, cv2.MORPH_CLOSE, kernel)`
- **Gradient hình thái học:** Tính toán độ dốc của hình dạng, tìm các biên → `cv2.morphologyEx(img, cv2.MORPH_GRADIENT, kernel)`
- **Top-hat:** Tìm các đối tượng sáng nhỏ trên nền sáng → `cv2.morphologyEx(img, cv2.MORPH_TOPHAT, kernel)`
- **Black-hat:** Tìm các lỗ hổng nhỏ trên nền tối → `cv2.morphologyEx(img, cv2.MORPH_BLACKHAT, kernel)`


```
import cv2
import numpy as np

cap = cv2.VideoCapture(0)

while(1):

    _, frame = cap.read()
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

    lower_red = np.array([30,150,50])
    upper_red = np.array([255,255,180])

    mask = cv2.inRange(hsv, lower_red, upper_red)
    res = cv2.bitwise_and(frame,frame, mask= mask)

    kernel = np.ones((5,5),np.uint8)
    erosion = cv2.erode(mask,kernel,iterations = 1)
    dilation = cv2.dilate(mask,kernel,iterations = 1)

    cv2.imshow('Original',frame)
    cv2.imshow('Mask',mask)
    cv2.imshow('Erosion',erosion)
    cv2.imshow('Dilation',dilation)

    k = cv2.waitKey(5) & 0xFF
    if k == 27:
        break

cv2.destroyAllWindows()
cap.release()
```

Danh mục tài liệu sử dụng:

- **Tài liệu học phần:** TS. Nguyễn Thị Khánh Tiên, *Chương 2: Các thuật toán xử lý ảnh (Part 2), Slide Học phần Xử lý ảnh & Thị giác máy tính*, Viện Công Nghệ Thông Tin, Điện & Điện Tử, Trường Đại Học Giao Thông Vận Tải Tp.HCM.
- **Tài liệu tham khảo:**
 - Trần Lê Anh (3/2025), *Chapter 2 – Part 3: Wavelet Transform & Geometric Transformations*, Viện Công Nghệ Thông Tin, Điện & Điện Tử, Trường Đại Học Giao Thông Vận Tải Tp.HCM.
 - Trần Đăng Nam, *Lecture Note Computer Vision*.
 - Các công cụ AI hỗ trợ.